



CS2023 - Programación III

Práctica Calificada #1b

Rubén Rivas Medina

Mayo 2025

Indicaciones Específicas

- **Tiempo límite:** 100 minutos
- **Formato de entrega:**
 - La solución debe implementarse en C++ y entregarse en tres archivos:
 - `ui_builder.h`, para la **pregunta #1**
 - `pipeline_apply.h`, para la **pregunta #2**
 - `compose.h`, para la **pregunta #2**
 - Los archivos deben nombrarse **EXACTAMENTE** como se indica (sensibles a mayúsculas)
- **Plataforma de entrega:**
 - Subir directamente los archivos a <https://www.gradescope.com> ó
 - Se puede crear un archivo comprimido `solucion.zip` que contenga o subirlos directamente:
 - `geodesy.h`, `geodesy.cpp`

Restricciones Clave

- La memoria dinámica puede gestionarse con punteros inteligentes: `std::unique_ptr` o `std::shared_ptr`

Recomendaciones a tomar en cuenta que podrían afectar su calificación

- **Asegurarse que cada nuevo metodo o atributo que agrega funcione.** a lo más podría fallar el ultimo método u operador implementado pero no todos, comentando el ultimo el resto funciona.
- **Asegurese de seguir las convenciones que se le solicitan,** nombres de clases, namespace, metodos exactamente igual a los solicitados en las pruebas, un uso incorrecto es sospecha del uso de un Chatbox y penalización en la nota.
- **Si la clase es totalmente inoperativa** (requiere cambios estructurales mayores para funcionar) **o contiene errores graves** (memory leaks críticos, crashes, implementación incorrecta de funcionalidad básica), La calificación máxima podría ser reducida hasta un **60%** del valor total de lo calificado.

Pregunta #1 Interfaz de Usuario (12 ptos)

Se desea construir un sistema de generación de interfaces gráficas de usuario (UI) que permita instanciar distintos tipos de widgets y aplicarles un estilo general mediante una política estática llamada Theme. Esta política de estilo debe decidirse en tiempo de compilación, mientras que los widgets concretos deben soportar despacho dinámico para permitir composiciones en tiempo de ejecución.

Requerimientos mínimos

1. Defina una clase base IWidget con un método virtual puro:

```
virtual void draw(std::ostream&) const = 0;
```

2. Implemente al menos dos clases derivadas:

- Button: con un campo label y salida en la forma [Button: OK].
- TextBox: con un campo content y salida en la forma [TextBox: .^Escriba aquí"].

3. Defina dos temas como estructuras sin estado:

- FlatTheme: imprime FLAT STYLE BEGIN y FLAT STYLE END.
- DarkTheme: imprime Dark Theme Begin y Dark Theme End.

4. Estilo típico de un tema:

```
struct nombre_de_theme {  
    static void pre_draw(std::ostream& os) {  
        // imprimir encabezado de estilo  
    }  
    static void post_draw(std::ostream& os) {  
        // imprimir pie de estilo  
    }  
};
```

5. Cree una clase plantilla ui_builder<Theme> que contenga:

- Un vector de punteros inteligentes a IWidget.
- Un método add<Widget>(...) que construya e inserte un widget.
- Un método render(std::ostream&) que imprima:
 - el encabezado de estilo (según el tema),
 - todos los widgets en orden,
 - el pie de estilo (según el tema).

Casos de prueba

A continuación se presentan algunos casos de prueba que deben ejecutarse correctamente si el sistema ha sido implementado de forma adecuada:

Caso 1: Tema plano con dos widgets

```
ui_builder<FlatTheme> ui;  
ui.add<Button>("Aceptar");  
ui.add<TextBox>("Ingrese texto");  
ui.render(std::cout);
```

Salida esperada:

```
FLAT STYLE BEGIN  
[Button: Aceptar]  
[TextBox: "Ingrese texto"]  
FLAT STYLE END
```

Caso 2: Tema oscuro con botón sin texto

```
ui_builder<DarkTheme> ui;  
ui.add<Button>("");  
ui.render(std::cout);
```

Salida esperada:

```
Dark Theme Begin  
[Button: ]  
Dark Theme End
```

Caso 3: Tema plano con múltiples botones

```
ui_builder<FlatTheme> ui;  
ui.add<Button>("OK");  
ui.add<Button>("Cancelar");  
ui.add<Button>("Salir");  
ui.render(std::cout);
```

Salida esperada:

```
FLAT STYLE BEGIN  
[Button: OK]  
[Button: Cancelar]  
[Button: Salir]  
FLAT STYLE END
```

Caso 4: Sin widgets

```
ui_builder<FlatTheme> ui;  
ui.render(std::cout);
```

Salida esperada:

```
FLAT STYLE BEGIN  
FLAT STYLE END
```

Caso 5: Tema oscuro con entrada compleja

```
ui_builder<DarkTheme> ui;  
ui.add<TextBox>("Usuario: \\root@localhost");  
ui.render(std::cout);
```

Salida esperada:

```
Dark Theme Begin  
[TextBox: "Usuario: \\root@localhost"]  
Dark Theme End
```

Asegúrese de que cada widget se imprima de forma independiente y en orden, y que los encabezados y pies de estilo se muestren correctamente según el tema utilizado.

Recomendaciones

- Aplique la Regla de los 5 en sus clases polimórficas.
- Evite copiar widgets, trabaje con `std::unique_ptr<IWidget>`.
- Puede utilizar templates variádicos en `add<Widget>(Args&&...)`.

Pregunta #2 Composición Genérica de Funciones

Implementar una funcionalidad genérica para aplicar múltiples funciones en secuencia sobre un valor inicial, así como componer funciones anidadas en un solo paso.

Parte 1: Aplicación secuencial (pipeline_apply)

Implemente una función plantilla: **pipeline_apply**. La función debe aplicar cada operación **op** en orden, tomando como entrada el resultado anterior. El valor inicial es **val**.

- Debe usar plegado sobre el operador de composición: (...).
- Debe retornar el resultado final.
- Debe aceptar funciones lambda, punteros a función o functors.

Ejemplo:

```
int result = pipeline_apply(3,
    [](int x) { return x * 2; },
    [](int x) { return x + 5; });
// result = (3 * 2) + 5 = 11
```

Parte 2: Composición de funciones (compose)

Implemente una función plantilla: **compose**, Esta función debe retornar un **lambda anónimo** que represente la composición de todas las funciones dadas, en orden inverso a la aplicación:

$$\text{compose}(f1, f2, f3)(x) = f1(f2(f3(x)))$$

- Debe capturar los argumentos por referencia perfecta.
- Debe usar una fold expression para aplicar las funciones.
- No es necesario que todas las funciones retornen el mismo tipo, pero sí que sean compatibles.

Ejemplo:

```
auto f = compose(
    [](int x) { return x + 1; },
    [](int x) { return x * 2; }
);
int r = f(5); // r = (5 * 2) + 1 = 11
```

Casos de pruebas de pipeline_apply**1. Secuencia aritmética simple**

```
auto f1 = [](int x) { return x + 1; };
auto f2 = [](int x) { return x * 3; };
int r = pipeline_apply(4, f1, f2); // Resultado esperado: 15
```

2. Transformación de tipos

```
auto to_str = [](int x) { return std::to_string(x); };
auto quote = [](const std::string& s) { return "\"" + s + "\""; };
std::string r = pipeline_apply(42, to_str, quote); // Resultado: "\"42\""
```

3. Sin operaciones

```
int r = pipeline_apply(99); // Resultado esperado: 99
```

4. Funciones con referencia

```
int value = 10;
auto add5 = [](int& x) { x += 5; return x; };
auto mul2 = [](int x) { return x * 2; };
int r = pipeline_apply(value, add5, mul2); // Resultado: 30
```

5. Uso de funciones constexpr

```
constexpr auto id = [](int x) { return x; };
constexpr int r = pipeline_apply(10, id, id, id);
static_assert(r == 10);
```

Casos de pruebas de compose

1. Composición simple f(g(x))

```
auto f = [](int x) { return x * 2; };
auto g = [](int x) { return x + 3; };
auto composed = compose(f, g);
int r = composed(5); // Resultado: 16
```

2. Composición con string

```
auto to_str = [](int x) { return std::to_string(x); };
auto parens = [](const std::string& s) { return "(" + s + ")"; };
auto bang = [](const std::string& s) { return s + "!"; };
auto exclaim = compose(bang, parens, to_str);
std::string r = exclaim(42); // Resultado: "(42)!"
```

3. Composición con una sola función

```
auto neg = [](int x) { return -x; };
auto id = compose(neg);
int r = id(7); // Resultado: -7
```

4. Composición con función mutable

```
int count = 0;
auto inc = [&count](int x) { ++count; return x + 1; };
auto c = compose(inc, inc, inc);
int r = c(5); // Resultado: 8, count == 3
```

Rúbrica de evaluación: ui_builder<Theme>(12 ptos)

Criterio	Ponderación	Descripción
Correctitud	60%	■ Renderizado correcto de widgets en orden ■ Aplicación del tema en encabezado y pie ■ Uso adecuado de <code>std::unique_ptr</code> y herencia ■ Despacho dinámico correcto de <code>draw()</code>
Robustez	30%	■ Soporte para temas alternativos ■ Añadir múltiples tipos de widgets sin error ■ No hay filtraciones de memoria
Legibilidad	10%	■ Código organizado con nombres descriptivos ■ Separación clara entre lógica del tema y lógica del widget ■ Comentarios donde aplica

Rúbrica de evaluación: pipeline_apply (4 ptos)

Criterio	Ponderación	Descripción
Correctitud	60%	■ Aplicación secuencial correcta de funciones ■ Soporte para valor sin funciones (retorna igual) ■ Deduce correctamente el tipo de retorno ■ Funciona con funciones que modifican por referencia
Robustez	30%	■ Funciona con tipos mixtos (<code>int</code>, <code>string</code>, etc.) ■ No hay errores de compilación con 0 o 1 función ■ No produce warnings por deducción o conversión
Legibilidad	10%	■ Uso claro de fold expressions ■ Estructura limpia y clara de la función ■ Comentarios cuando son necesarios

Rúbrica de evaluación: compose (4 ptos)

Criterio	Ponderación	Descripción
Correctitud	60%	■ Composición correcta de funciones en orden inverso ■ Resultado correcto para 1, 2 o más funciones ■ Funciona con tipos distintos entre funciones ■ Función retornada es invocable correctamente
Robustez	30%	■ Soporta capturas mutables ■ Funciona como expresión lambda sin errores ■ No produce warnings al combinar tipos
Legibilidad	10%	■ Código expresivo y sin duplicación ■ Captura de lambdas clara y ordenada ■ Buen uso de perfect forwarding si aplica